RETURN-ORIENTED PROGRAMMING: ANALYSIS AND EXPLOITATION OF A GAME

LOVIS SCHUNTER *LAURIN VELLMETE* *FLORIAN KRASNIQI*

BINARY HACKING

UNIVERSITY HEIDELBERG

TABLE OF CONTENTS

ABSTRACT

A common goal in exploits is the execution of code that the binary did not intend. A classic approach is code injection, in which the attacker tries to inject his own code into the binary. But code injection is quite hard and often secured by many mechanisms. In return oriented programming, short rop, the attacker does not inject new code but instead uses existing code in a rearranged order to achieve new semantics. This is done by highjacking the control flow and injecting the addresses of existing code. We developed a simple maze-like game that is vulnerable to a rop attack and then implemented cheats through rop.

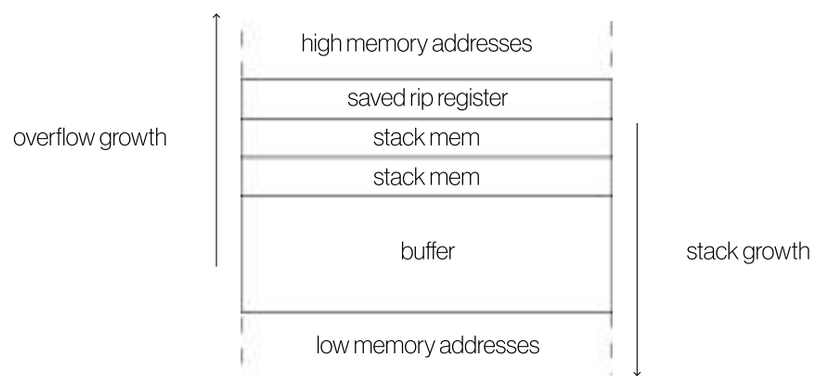
ROP EXPLOITATION *BACKGROUND*

The first introduction to rop was done by a researcher at the University of California San Diego, named Hovav Shacham, who in his paper “The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)”[1] considered the feasibility of return-to-libc attacks with no function calls.[2]

In this write up a look at the theoretical foundations as well as practical uses will be taken. Generally speaking rop attacks can have many points of entry, but the main focus will be on a stack buffer overflow. To understand the location of the attack one needs to understand the stack layout of a common function. In this write-up the focus will be on the x86 architecture in 32 or 64 bit, but rop does work on other architectures too. A basic function might look like this[3]

```
int main(int argc, char** argv) {
    char buffer [100];
    strcpy ( buffer , argv [1])
}
```

while the stack layout might look like this



The first goal in a return oriented programming attack is to highjack the stack address where the return register is (rip in

x86_64 and eip in x86) to highjack the control flow of the execution upon return.

This can be achieved by a stack buffer overflow, where the buffer is overflown to the point where the rip is being overwritten, the distance between the beginning of the stack and the point where the rip starts is referred to as the offset. After the offset the address of the desired code snippet can be injected - from that point the ROP chain starts. The attacker can now append addresses of code snippets to get a new semantic and essentially inject a new block of code, not by creating new code but by patching together existing one. In the other chapters a closer look at the specifics will be taken.

Now for some more theoretical background and why rop is so interesting to attackers. As said earlier rop allows the attacker to create new code without the need for code injection, this is particularly interesting because of the data execution prevention (short DEP) that Microsoft introduced, which prevents exactly this kind of new code injection, rop bypasses that.

Another interesting aspect is the turing completeness of rop attacks. Classic code injection is appealing because the attacker can implement any code he wants, but is the same achievable with rop? A paper from UC touched on the aspect that rop with a certain set of code snippets can be turing complete[4], and often the standard libraries included in most C / C++ code like libc is sufficient for a turing complete set of code snippets. So that implies that with a rop attack and a particular set of code snippets, basically every block of code could be achieved semantically.

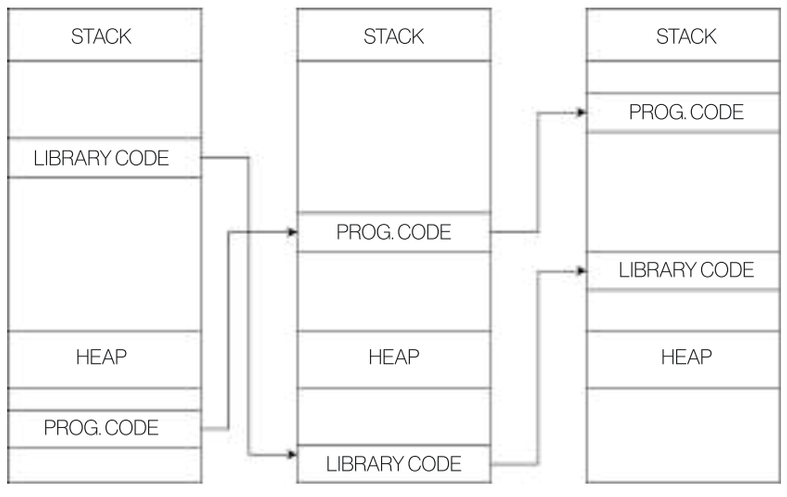
REQUIREMENTS FOR ROP *BACKGROUND*

In general, requirements are not set in stone, some of them can be bypassed or are subject to zero-day attacks.

But a few requirements should be met, or at least make it easier in a rop attack. The attacker needs some kind of vulnerability, as stated earlier this must not be a buffer overflow, it could be a format string vulnerability, a heap overflow, memory corruption and so on. To address everything would go beyond the scope of this write-up, it is sufficient to state that that there needs to be one.

Because the attacker needs to highjack the control flow, there are a few security mechanisms that need to be disabled, such as stack protection.

Now for some memory requirements, (modern) systems have address space layout randomisation, short aslr. Aslr is a mechanism that randomises the memory on every execution of the most important code components, which might look like this:



While there are some approaches to rop that allow an attack with aslr in place, it is very hard, and so the usual attack is on a system where aslr is deactivated.

If the attacker is working with a stack overflow, he might encounter stack canaries or stack cookies, those are a fixed part of the stack, which is checked for overflow (if it has been altered) and so the program crashes. So to allow for an overflow, stack protection such as stack canaries must be disabled.

When trying to highjack the control-flow one might encounter mechanisms that prevent this, one of those are 'endbr64' instructions which check it the control-flow has been altered. While it is hard to fully disable those systems, the occurrence of such instructions can be reduced significantly.

GADGETS *BACKGROUND*

In this write up, there was a common reference to "code-snippets", these code-snippets are called gadgets. Gadgets are small assembly code snippets that always end with a return statement. With the return-statements an insurance is given, that after the gadget it jumps back and then into the next gadget. A rop-chain, the core of a rop attack, consists of a set of addresses that lead to gadgets. Lets inspect a small rop chain, in x86 (32-bit) that summons a shell, here the gadgets will be chained together not the addresses for illustration purposes

```
pop ebx
ret
address of /bin/sh
xor ecx, ecx
ret
xor eax, eax
ret
int 0x80
ret
```

This chain pops the stack, where the address of /bin/sh lies, nullifies the registers ecx and eax and then executes a syscall with the 0x80 interrupt. Immediately after a shell in the current privilege is summoned.

Gadgets can be extracted from many places, from the binary itself as well as the libraries. As touched upon above, almost every C program uses some kind of libc, which include a huge variety of gadgets.

GOAL OF OUR ROP ATTACK *METHODOLOGY*

In our case we have a buffer overflow vulnerability, which opens the binary up to a lot of exploits, in our case a rop attack. From here there are many, if not infinite possibilities.

We focus on changing a variable, lets take a closer look

```
class Camera {
private:
    float                   _zoom;
    ...

public:
    Camera(float zoom, Player* player, SDL_Rect* playerRect);
    ...
};
```

As we will discuss later in implementation, the Camera class handles the camera, so the part of the image that the user sees. It can have to following effect on the game

As soon as the '_zoom' member variable changes the view changes too. This member variable has a fixed place in the stack, so it is vulnerable to our rop attack.

RETRIVIAL OF GADGETS AND ADRESSES

METHODOLOGY

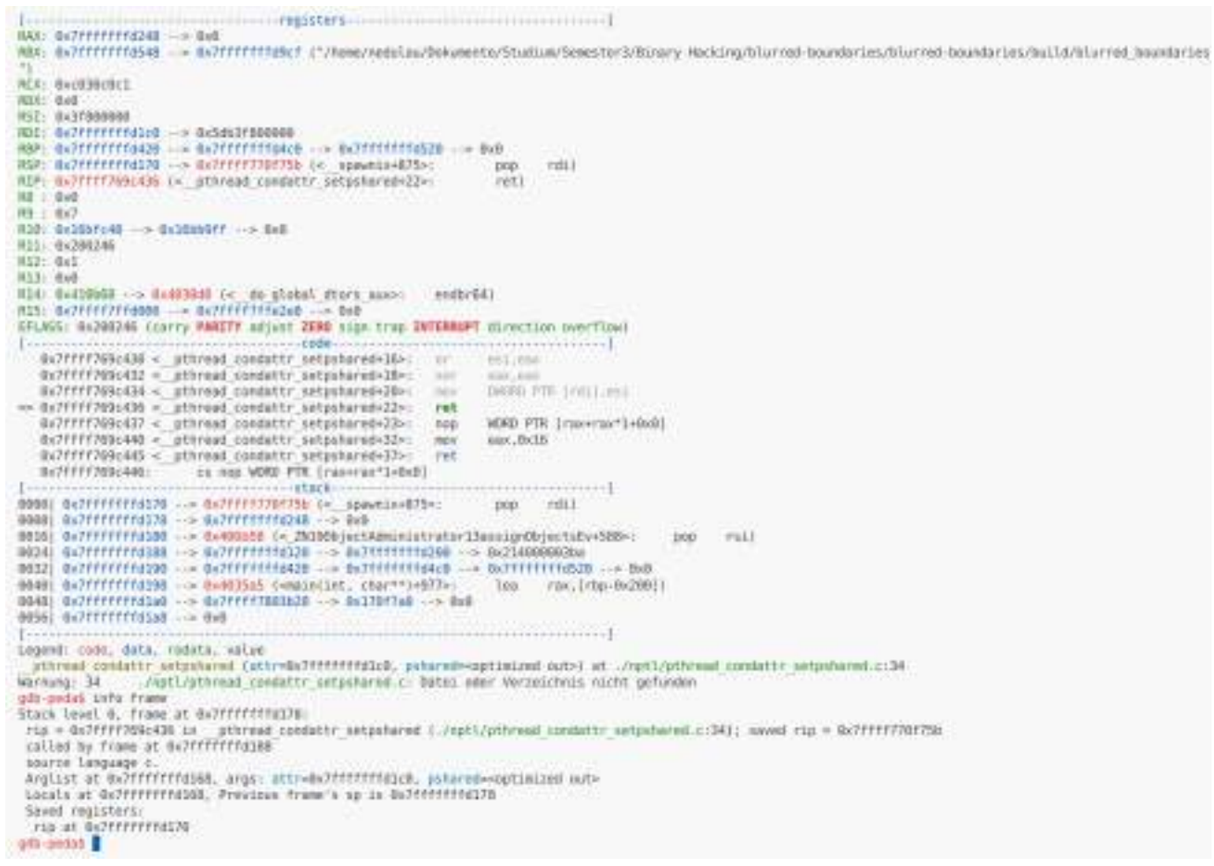
The plan of the attack is to load the value of 1.0f in float formatting into a register and the address if the '_zoom' variable into another and then use a pointer to load the value into the member variable, the rop chain might look like this

```
pop rdi
ret
0x000000003F800000 ; Float formatting of 1.0f
pop rsi
ret
0 x7fffffffd1c0   ; Example Address of _zoom
mov dword ptr [rdi] esi
ret
0x0000555555557f19 ; Example Address to return in regular flow
```

So we load 1 into rdi, load the address of '_zoom' into rsi and then load the lower 16 bits (all a float needs) into

'_zoom' through a pointer.

Now we come to a more pratical part and here we need to highlight gdb-peda, an extension of gdb. The interface of gdb peda looks like this



The first step was to overflow the buffer, this is not, like one might intuitively think just the buffer size but an offset consisting of the range until we reach the saved rip register. We used a cyclic pattern, that can be created by the command 'pattern create <size>'. Usually this part is quite simple, we just need the value of the pattern in our saved rip register and check the offset size with 'pattern offset <pattern-fragment>', in our case it turned out to be a bit more difficult as the value was nested in another stackframe then displayed by default. But with quick 'backtrace' and the selection of the frame we could see our value easily displayed and even labeled as 'saved-rip'. Now we had the offset, which was 40.

The next step was the general adress retrival of the needed gadgets, here we used a tool called ROPGagdets, which extracted all of the gadgets in a binary into a textfile, the view of the textfile looked like this

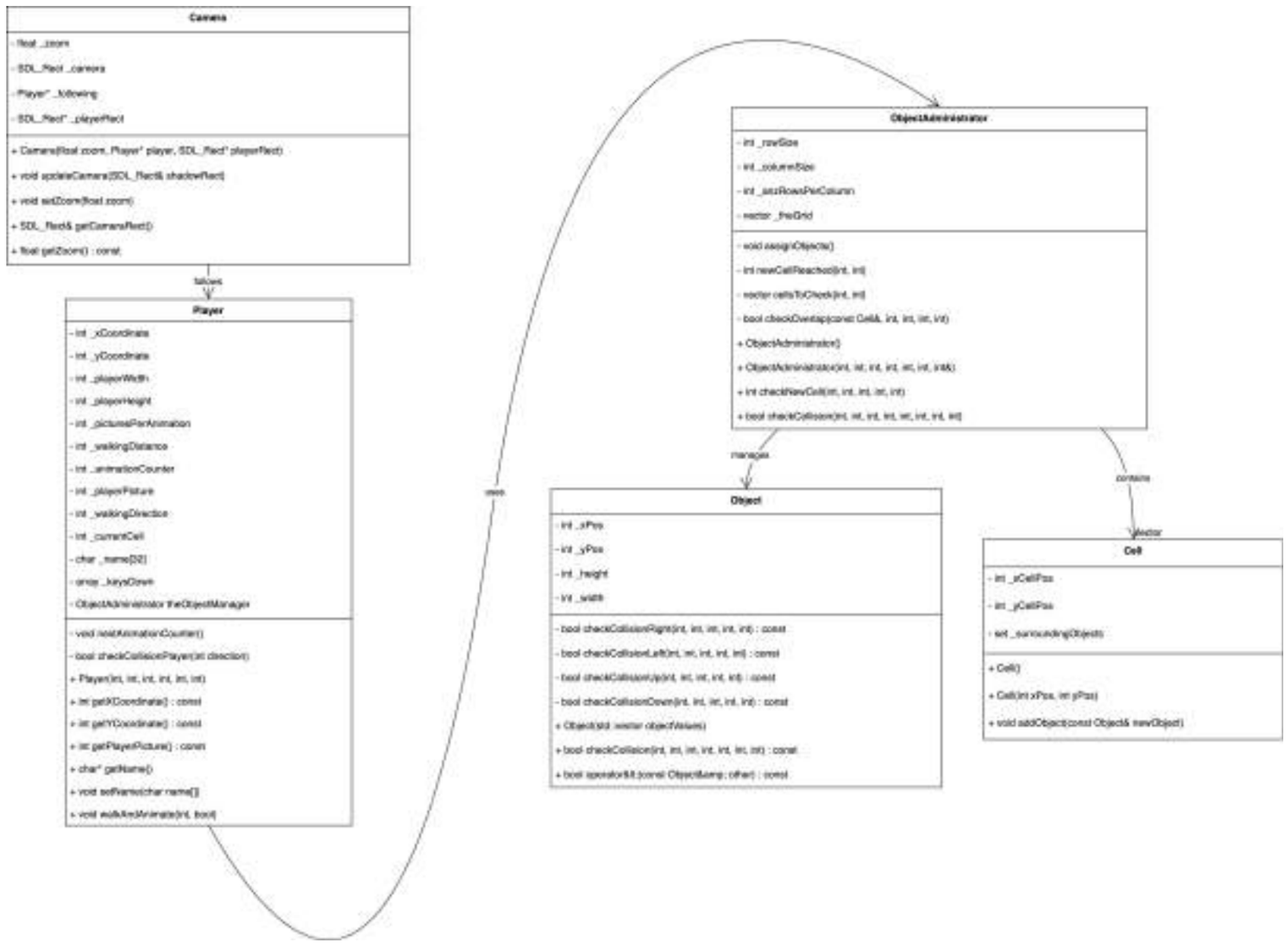
```
Gadgets information
============================================================
0x0000000000003175 : adc al, 0 | add byte ptr [rax], al | jmp 0x3020
0x00000000000046d7 : adc al, 0x29 | ret 0x428d
0x0000000000005951 : adc al, 0x29 | ret 0x8b48
0x00000000000040a0 : adc al, 0x89 | ret 0xeac1
0x0000000000005a0a : adc al, 1 | ret 0x8b48
0x0000000000003155 : adc al, byte ptr [rax] | add byte ptr [rax], al
                     jmp 0x3020
                                ...
```

As stated earlier gadgets can also be extracted from libc, which we did, here the base address needs to be added.
Now our payload was ready to be constructed. We created it using the 'struct.pack("<Q", <address in hex>)' from the pwn lib in Python which packed our hex addresses in proper formatting for a little endian system like x86.

THE GAME LIB SDL2 *IMPLEMENTATION*

The game library, simple direct media-layer, short SDL, helped us to develop the underlying game. SDL is very easy to use and has a platform-independent API. Also it is very well maintained, while developing our game SDL3 was released.

INTRODUCTION TO OUR CLASSES *IMPLEMENTATION*



We adapted the following class model. The Camera class manages as already said, the part of the rendered screen that the player sees. The Player class manages player-movement, animation and other details. The biggest class is our collision detection class, called ObjectAdministrator. It is our custom collision detection, that works well with our style of map and game implementation, because non of the existing solutions worked.

DETAILS OF IMPLEMENTATION *IMPLEMENTATION*

```
void setName(int argc, char* argv[], Player* player) {
    char name[32];

    if (argc > 1) {
        // name from command line argument
        strcpy(name, argv[1]);
    } else {
        // name from file
        read(STDIN_FILENO, name, 1000);
    }

    player->setName(name);
}
```

Worth mentioning is the place where the buffer overflow happens, in the setName() function, the buffer can be overflown through command-line-arguments or files.

```
void Camera::updateCamera(SDL_Rect& shadowRect) {
    ...

    // calculate ideal camera position to center on player
    int idealCamX = playerCenterX - (_camera.w / 2);
    int idealCamY = playerCenterY - (_camera.h / 2);

    // apply camera bounds
    _camera.x = std::max(0, std::min(WINDOW_WIDTH - _camera.w,
        idealCamX));
    _camera.y = std::max(0, std::min(WINDOW_HEIGHT - _camera.h,
        idealCamY));

    // calculate player's screen position relative to camera, accounting
        for zoom
    _playerRect->x = (_following->getXCoordinate() - _camera.x) * _zoom;
    _playerRect->y = (_following->getYCoordinate() - _camera.y) * _zoom;
    ...
}
```

Another interesting aspect is the centering of the camera, it need to always follow the player and prevent crossing world borders, we came up / used this code to center the camera relative to the player while respecting bounds

```
int checkNewCell(int playerXPos, int playerYPos, int playerWidht, int
    playerHeight, int pIdxCell);

bool checkCollision(int playerXPos, int playerYPos, int playerWidht, int
    playerHeight, int xMovement, int yMovement, int direction, int idxCell);
```

The ObjectAdministrator class contains several noteworthy aspects of implementation.

One of the most interesting details is the grid structure used for collision detection. Each cell in the grid holds a set of objects, and the grid is initialized globally while collisions are checked locally. When the player moves far enough to enter a new cell, the current cell is updated. During movement, collisions are tested against objects in the player's current cell and the three adjacent cells, depending on the direction of movement.

CONCLUSION

In our analysis of rop many interesting aspects surfaced. As soon as the rip is highjacked in any way, a huge set of options is available, and that with no further code injection. In theory it would pose a risk, but it seems to be one of those zero-day attacks that academia discovered before any major attacks, as there seems to be no popular example of rop used to harm. Rop is not to be underestimated though, because as soon as the attacker highjacks the control flow its a simple matter of acquiring the addresses of gadgets and chaining them.

In this write up we explored the theoretical example of a vulnerable executable to rop and laid down the conceptional approach to attacking it. Our game is an example on how rop can be used in a creative and illustrative way to cheat.

REFERENCES

[1] The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86): https://hovav.net/ucsd/dist/geometry.pdf

[2] Arm Exploitation through ROP: https://blog.3or.de/arm-exploitation-return-oriented-programming

[3] ROP Chaining: Return Oriented Programming: https://www.ired.team/offensive-security/code-injection-process-injection/binary-exploitation/rop-chaining-return-oriented-programming

[4] Microgadgets: Size Does Matter In Turing-complete Return-oriented Programming: https://www.usenix.org/system/files/conference/woot12/woot12-final9.pdf